

Exploring the use of aligned and disaligned multimodal features in MultiModal Recommender Systems

Fontana Emanuele

January 8, 2026

Contents

1	Introduction	3
1.1	Recommender Systems	3
1.2	Multimodal Recommender Systems	4
2	Aligned multimodal models	6
2.1	Contrastive Learning	6
2.1.1	CLIP	6
2.1.2	AudioCLIP	7
2.2	Contrastive Learning arcitetures in Multimodal Recommender Systems	8
2.2.1	CLIP in Multimodal Recommender Systems	8
2.2.2	AudioCLIP in Multimodal Recommender Systems	10
3	Experimental Settings	11
3.1	Research Questions	11
3.2	Datasets	11
3.2.1	MovieLens 1M	11
3.2.2	LastFM	11
3.3	Multimodal Embedding Extraction Pipeline	12
3.3.1	Overview	12
3.3.2	File Organization and Naming Convention	12
3.3.3	Embedding Models	13
3.3.4	Extraction Process	13
3.4	Interaction Filtering and Remapping Pipeline	14

3.4.1	Common Pipeline Steps	14
3.4.2	MovieLens Mapping	14
3.4.3	LastFM Mapping	15
3.5	Data Distribution Extraction	16
3.5.1	Overview	16
3.5.2	UMAP Dimensionality Reduction	16
3.5.3	Density Estimation	17
4	Experiments	19
4.1	Data distribution	19
4.1.1	MovieLens 1M	19
4.1.2	LastFM	23
4.2	Results Overview	25
4.2.1	MovieLens_1M	26
4.2.2	LastFM	30
5	Discussion	33
5.1	Data Distribution Analysis	33
5.2	Performance Analysis	33
5.2.1	Model Comparison	34
5.2.2	Impact of Feature Alignment on MovieLens 1M	34
5.2.3	Impact of Feature Alignment on LastFM	35
5.2.4	The Role of Audio Modality	35
6	Conclusion and Future Work	36
6.1	Summary of Findings	36
6.2	Interplay between Model Complexity and Data Alignment	36
6.3	Future Work	37

1 Introduction

1.1 Recommender Systems

With **Recommender Systems** we refer to all those techniques that have the effect of guiding users in personalized way to interesting objects in a large space of possible options[5]. There are different types of Recommender Systems:

- **Content-Based Filtering:** This approach recommends items similar to those the user has liked in the past. It uses item features and user preferences to make recommendations.
- **Collaborative Filtering:** This method recommends items based on the preferences of similar users. It can be user-based or item-based.
 - **User-Based Collaborative Filtering:** This method finds users with similar preferences and recommends items they have liked.
 - **Item-Based Collaborative Filtering:** This method finds items similar to those the user has liked and recommends them. With respect to Content-Based Filtering, this method doesn't use item features to make recommendations.
- **Hybrid Methods:** These methods combine multiple recommendation techniques to improve accuracy and overcome the limitations of individual approaches.
- **Knowledge-Based Recommender Systems:** These systems use explicit knowledge about users and items to make recommendations.
- **Context-Aware Recommender Systems:** These systems take into account contextual information, such as time, location, or social context, to provide more relevant recommendations.

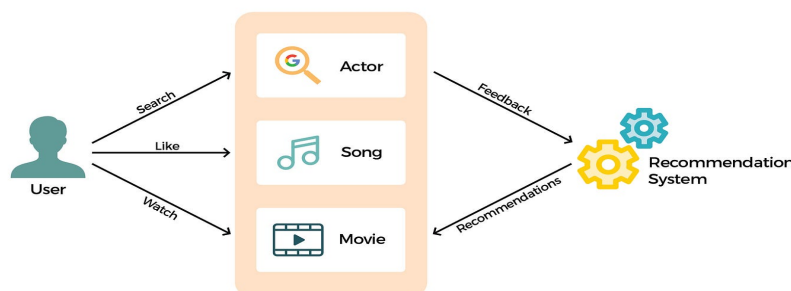


Figure 1: Conceptual representation of a Recommender Systems

Recommender Systems are not perfect. They can suffer of several problems like:

- **Cold Start Problem:** Because new users often have very few records in the system, it is hard to guess their preferences given the insufficient information[8]
- **Data Sparsity:** Most users use the system but do not give rating for feedback to the system in a proper way.[6]. In addition, in many real-world application we have thousand if not milion of user and item, so it's impossible to think that all user with interact with all item.
- **Scalability:** As the number of users and items increases, the computational complexity of recommendation algorithms can become a challenge. So we can divide scalability problems in two categories:
 - **Software Scalability:** There's a need to develop algorithms that can handle milions of users and items
 - **Hardware Scalability:** There's a need to create systems with powerful hardware that can handle the computational cost in time and space
- **Over Specialization :** Sometimes Recommender algorithms can "over-fit" users' preferences, which lead to suggest items that are too similar to those already liked by the user not allowing him/her to discover new items that might be relevant

1.2 Multimodal Recommender Systems

Differently from classical recommendation approaches, multimodal recommendation exploits multimodal side information about items to enrich the interaction matrix, alleviating its sparsity, and to better understand the content to be recommended; these advantages, in general, lead to more accurate and precise recommendations[9]. We can think about different types of multimodal information: **Text** in natural language that can be used in every context, **Audio** for Music, **Images** for products such as clothes or furnitures, **Videos** for movies and so on.

The powerfulness of multimodal recommendation is given by the fact that different modalities can provide information about the same item from different perspectives. For example, in a movie recommender system, the textual description of a movie can provide information about its plot, screenshots

from the film can provide visual clues about the style of photography and the type of film (animated, live-action, stop-motion, etc.)

Some DNN architectures known as **Encoders** are used to extract features from different modalities that can be used in different ways like:

- **Concat:** Refers to the concatenation of multimodal features. [1]
- **Fusion and attention:** Refers to the attention mechanisms (like self-attention) to combine multimodal features. [4]
- **GNNs:** They use Graph Neural Networks to model the relationships between items and users, incorporating multimodal features into the graph structure

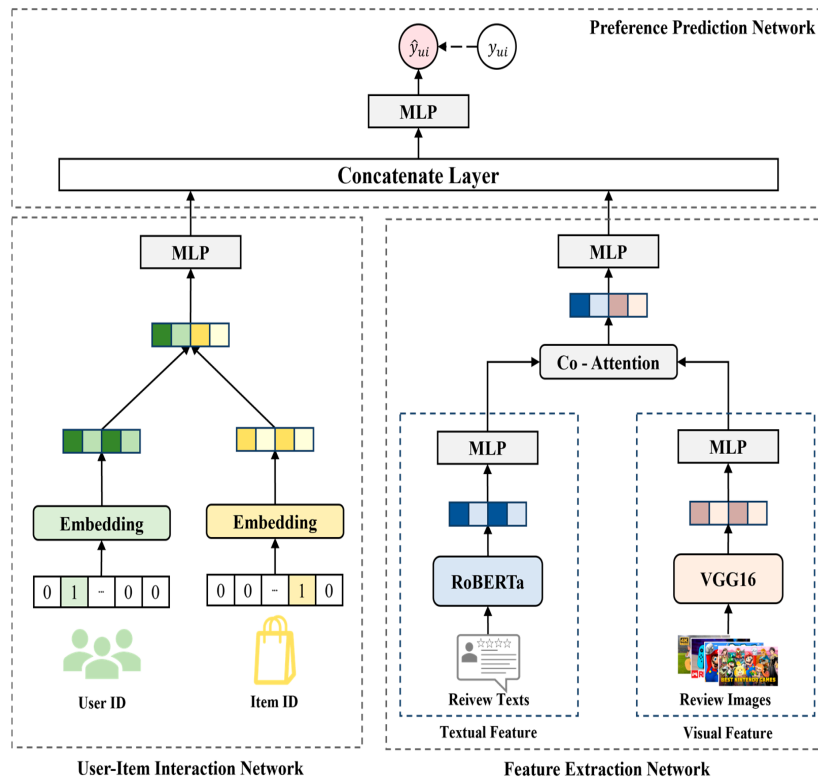


Figure 2: Conceptual representation of a Multimodal Recommender Systems

2 Aligned multimodal models

2.1 Contrastive Learning

Contrastive Learning is self-supervised representation learning by training a model to differentiate between similar and dissimilar samples.[3]. In few words, contrastive learning is a technique used to learn a feature space where similar samples are close together and dissimilar samples are far apart. To achieve this, a specific loss function known as **Contrastive Loss** is used:

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} [k \neq i] \exp(\text{sim}(z_i, z_k)/\tau)}, \quad (1)$$

where $\text{sim}(z_i, z_j)$ represents the similarity between two feature vectors z_i and z_j , τ is a temperature parameter, and N is the batch size. The numerator focuses on the similarity between the positive pair (z_i, z_j) , while the denominator normalizes over all pairs excluding z_i itself.

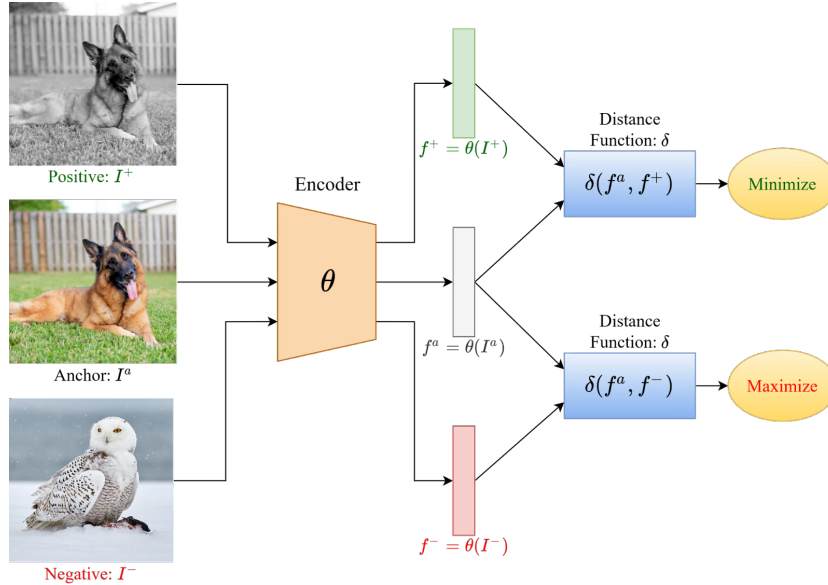


Figure 3: Conceptual representation of Contrastive Learning

2.1.1 CLIP

CLIP, which stands for Contrastive Language-Image Pre-training, is a neural network architecture developed by OpenAI[7] that uses Contrastive Learning to connect textual and visual information. The main idea behind CLIP is that visual and textual embeddings live in the same feature space. Given

the textual description of an image and the image itself, the embeddings returned by the model should be close together in the feature space. In few words, CLIP is trained to predict which images are most relevant to a given text prompt, and vice versa.

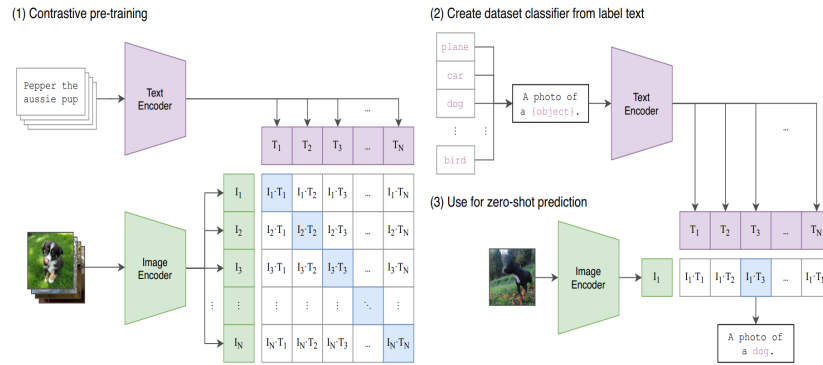


Figure 4: Conceptual representation of CLIP architecture

2.1.2 AudioCLIP

AudioCLIP [2] is an extension of the CLIP model that incorporates an additional audio modality, enabling it to understand and relate audio data alongside text and images. The architecture of AudioCLIP consists of three main components:

- **A CLIP Model** to handle text and image modalities. In particular a ResNet based CLIP model is used
- **Audio Encoder** to handle audio modality. In particular an ESResNetXt model is used

The idea to realize AudioCLIP is very simple: in addition to the text-image contrastive loss used in CLIP, two additional loss were added: **text-audio** and **image-audio** contrastive loss. In this way the three modalities are connected in the same feature space. The training procedure of AudioCLIP consists of different main steps:

- **Audio-Head Pre-Training**
 - **Standalone:** The audio head (based on ESResNeXt) is first pre-trained independently on the *AudioSet* dataset.
 - **Cooperative:**

- * The classification layer of the pre-trained audio head is replaced with a randomly initialized layer whose output size matches CLIP’s embedding space.
- * The audio head is then trained jointly with the frozen text and image heads, in a *multi-modal knowledge distillation* setup, making audio embeddings compatible with CLIP embeddings.

• AudioCLIP Training

- After making the audio head compatible with CLIP, the entire tri-modal model (audio, text, image) is trained on *AudioSet*.
- All three modality-specific heads are updated together, allowing the model to adapt to the distributions of audio samples, image frames, and textual class names.
- This joint training improves performance compared to training only the audio head.

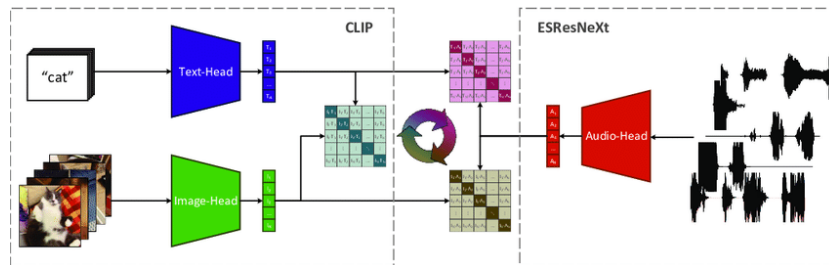


Figure 5: Conceptual representation of AudioCLIP architecture

2.2 Contrastive Learning arcitetures in Multimodal Recommender Systems

CLIP and AudioCLIP were born, and mainly used, for classification tasks. However, their ability to create a shared feature space for different modalities makes them suitable for multimodal recommender systems as well.

2.2.1 CLIP in Multimodal Recommender Systems

Zixuan Yi et al. [10] analyzed the use of CLIP in multimodal recommender systems. Up to few years ago, multimodal features were extracted using pretrained models on a single modality, such as ResNet for images and BERT for text. This leads to a problem: the features extracted from different

modalities are not aligned in the same feature space. To solve this problem, they proposed to use CLIP to extract aligned image and text features. In particular, they used both a frozen and a fine-tuned CLIP model to extract image and text features. Experiments involved five multimodal recommender systems: VBPR, MMGCN, MMGCL, SLMRec and LATTICE and showed that both frozen and fine-tuned CLIP features significantly outperformed the traditional pretrained features in four out of five models. To be more specific, due to its learning objective, LATTICE performs better without CLIP features. The fine-tune algorithm proposed by the authors is known as **END-TO-END Training**.

End-To-End Training The end-to-end fine-tuning procedure integrates the CLIP encoder directly into the recommendation model and jointly optimizes both components using recommendation losses. The training steps are as follows:

1. **Load Data:** Prepare the dataset by loading raw data
2. **Initialize CLIP Encoder:** Load a pre-trained CLIP model and its corresponding weights.
3. **Generate Embeddings:** Use the CLIP encoder to extract image embeddings and text embeddings from the dataset:
4. **Integrate Embeddings:** Feed the extracted embeddings into the recommendation model as initial item representations.
5. **Joint Optimisation:** For each training epoch, jointly update both the CLIP encoder and the recommendation model:
 - (a) Perform a forward pass to compute user-item scores.
 - (b) Compute the recommendation loss
 - (c) Backpropagate the loss and update both the CLIP encoder and recommendation model parameters:
6. **Evaluation:** After each epoch, evaluate and log recommendation performance metrics (e.g., NDCG, Recall) to monitor progress.

This end-to-end strategy allows the CLIP encoder to adapt its visual and textual representations specifically to the recommendation task, leading to improved alignment between modalities and better overall recommendation performance.

2.2.2 AudioCLIP in Multimodal Recommender Systems

In literature there are no evidence of the use of AudioCLIP in multimodal recommender systems.

3 Experimental Settings

3.1 Research Questions

The main research questions that this project aims to address are:

- **RQ1: Qualitative Analysis (Geometry)**
How does the alignment process (e.g., CLIP, AudioCLIP) affect the geometric distribution of the feature space?
- **RQ2: Quantitative Analysis (Performance)**
Can the integration of *aligned* multimodal embeddings (text, image, audio) enhance recommendation accuracy compared to unimodal or non-aligned approaches?

3.2 Datasets

This project employs two benchmark datasets for evaluating multimodal recommendation:

3.2.1 MovieLens 1M

MovieLens 1M is a widely-used collection of explicit movie ratings containing **User-movie interactions**: implicit feedback from user ratings. For each movie, the dataset has been enriched with multimodal data:

- **Images**: movie posters (.jpg format)
- **Audio**: movie trailer (.wav format)
- **Text**: textual descriptions, synopses, or reviews (.txt format)

3.2.2 LastFM

LastFM is a music listening dataset containing:

- **User-artist interactions**: implicit feedback from listening history
- **Multiple tracks per artist**: track-level multimodal features

For each track, the dataset includes:

- **Images**: album artwork (.jpg format)
- **Audio**: songs (.wav format)

- **Text:** track metadata or descriptions (.txt format)

Note: LastFM presents a unique challenge where interactions are recorded at the artist level, while multimodal features are extracted at the track level. This mismatch is addressed in the mapping pipeline (Section 3.4).

3.3 Multimodal Embedding Extraction Pipeline

The embedding extraction pipeline is **dataset-agnostic** and applies the same process to both MovieLens and LastFM. This section describes the unified extraction methodology.

3.3.1 Overview

The extraction pipeline performs the following steps:

1. File discovery and alignment across modalities
2. Validation and corruption detection
3. Per-modality embedding extraction using multiple encoder architectures
4. Strict consistency enforcement across modalities
5. Output generation in aligned NumPy arrays

3.3.2 File Organization and Naming Convention

All multimodal files are organized with a consistent basename structure. For both datasets:

- Each item (movie/track) is identified by a unique ID
- Files across modalities share the same basename

Example (MovieLens): For movie ID 1:

- Image: `ml1m/_images/1.jpg`
- Audio: `ml1m/_audios/1.wav`
- Text: `ml1m/_texts/1.txt`

Example (LastFM): For artist 1000 with track variant 1:

- Image: `lastfm/_images/1000_1.jpg`
- Audio: `lastfm/_audios/1000_1.wav`
- Text: `lastfm/_texts/1000_1.txt`

(there are up to 5 songs per artist)

3.3.3 Embedding Models

Multiple encoder architectures are employed to extract embeddings, providing both aligned and specialized representations:

Aligned Multimodal Models

- **AudioCLIP**: Extracts aligned embeddings across all three modalities (image, audio, text) in a shared feature space
- **CLIP**: Extracts aligned image-text embeddings

Modality-Specific Models

- **ViT (Vision Transformer)**: Image embeddings
- **VGGish**: Audio embeddings
- **MiniLM**: Text embeddings

3.3.4 Extraction Process

Step 1: File Discovery and Alignment Directories containing raw multimodal data are scanned:

- Identifies files by basename (e.g., `movie ID` or `artist_track` combination)
- Computes the **intersection** of basenames across all three modalities

Only items present in all three modalities are considered candidates for extraction. This ensures row-wise correspondence in the final embedding arrays.

Step 2: Extraction Files that pass validation are processed in order to extract embeddings. If the extraction for a specific modality fails (e.g., due to file corruption), the entire item is excluded to ensure strict alignment.

3.4 Interaction Filtering and Remapping Pipeline

After embedding extraction, the interaction datasets must be filtered and remapped to ensure consistency with the available multimodal features. This section describes the **dataset-specific** mapping procedures.

3.4.1 Common Pipeline Steps

Both datasets follow the same core logic:

1. Load original interaction file and a .csv mapping of item IDs to embedding row indices
2. Apply 5-core filtering to ensure interaction density
3. Remap user and item IDs to contiguous ranges $[0, \dots, n-1]$
4. Reconstruct embedding arrays in the new item order
5. Generate output files with remapped IDs and embeddings

The key difference lies in **Step 2**, where LastFM requires special handling for artist-to-track mapping.

3.4.2 MovieLens Mapping

Step 1: Loading Data The script loads:

- `movielens_1m.inter`: Original interaction file (user_id, item_id, rating, timestamp)
- A .csv mapping from movie ID to embedding row index
- All .numpy embedding files

Step 2: Feature-Based Filtering Check between `movielens_1m.inter` and the .csv file to retain only interactions referencing movies contained in both files.

Step 3: 5-Core Filtering To address data sparsity, the script applies iterative 5-core filtering:

Algorithm:

1. Count interactions per user and per item

2. Remove users with < 5 interactions
3. Remove items with < 5 interactions
4. Repeat until convergence (no further removals)

This ensures that every user has rated at least 5 items and every item has been rated by at least 5 users.

Step 4: ID Remapping After filtering, IDs may be non-contiguous. The script creates contiguous mappings:

- **Users:** $\{u_1, u_2, \dots, u_m\} \rightarrow \{0, 1, \dots, m - 1\}$
- **Items:** $\{i_1, i_2, \dots, i_n\} \rightarrow \{0, 1, \dots, n - 1\}$

Step 5: Embedding Reindexing After remapping, the embedding arrays are reconstructed to match the new item order and a new .csv mapping file is generated.

3.4.3 LastFM Mapping

LastFM requires special handling due to a structural mismatch:

- **Interactions** are recorded at the **artist level** (user listens to artist)
- **Embeddings** are extracted at the **track level** (one feature set per track)

Resolution Strategy: Artist-to-Track Expansion The mapper resolves this mismatch by:

1. Mapping each artist to all available track variants in the .csv file
2. Expanding each user-artist interaction into multiple user-track interactions
3. Treating each variant as a separate item in the recommendation task

Step 1: Loading Data The script loads:

- **lastfm.inter:** Original interaction file (user_id, artist_id, listen_count, timestamp)
- A .csv mapping from **artist_track** identifier to embedding row index
- All .numpy embedding files

Step 2: Artist-to-Track Mapping and Expansion The script performs two sub-steps:

Sub-step 2a: Build Artist-to-Tracks Dictionary

- Parse the .csv file to extract artist and track identifiers
- Group tracks by artist: $\text{artist} \rightarrow [\text{track}_1, \text{track}_2, \dots]$

Sub-step 2b: Expand Interactions For each user-artist interaction :

- Lookup available tracks for artist a : $T_a = [\text{track}_1, \dots, \text{track}_k]$
- Create k new interactions: $(u, \text{track}_1), \dots, (u, \text{track}_k)$

This expansion significantly increases the number of interactions.

Step 3–5: Standard Pipeline After expansion, the pipeline follows the same steps as MovieLens

3.5 Data Distribution Extraction

After extracting and remapping embeddings, it is essential to analyze their geometric and statistical properties to understand how different encoders structure the feature space.

3.5.1 Overview

The distribution extraction pipeline performs the following steps:

1. Load pre-extracted embedding arrays (.npy files)
2. Apply UMAP for dimensionality reduction to 2D
3. Normalize reduced embeddings onto the unit circle (S1 manifold)
4. Compute 2D Gaussian KDE for spatial density visualization
5. Compute angular KDE for directional distribution analysis

3.5.2 UMAP Dimensionality Reduction

High-dimensional embeddings are reduced to 2D using UMAP (Uniform Manifold Approximation and Projection).

Algorithm Overview UMAP is technique used to projects an high-dimensional space into a lower-dimensional soace while preserving both local and global structure. The algorithm operates in three main phases:

1. **Graph Construction:** Build a weighted k-nearest neighbor graph in high-dimensional space, where edge weights represent fuzzy membership strengths based on local distance distributions
2. **Fuzzy Simplicial Set:** Convert the neighbor graph into a fuzzy topological structure that captures the geometry
3. **Optimization:** Minimize the cross-entropy between the high-dimensional fuzzy set and its low-dimensional representation using stochastic gradient descent

S1 Normalization After UMAP reduction, all points are projected onto the unit circle:

1. Center the data: $\mathbf{x}'_i = \mathbf{x}_i - \bar{\mathbf{x}}$
2. Normalize by Euclidean norm: $\mathbf{x}''_i = \frac{\mathbf{x}'_i}{\|\mathbf{x}'_i\|}$

3.5.3 Density Estimation

Two complementary density estimation methods are applied to quantify the spatial and directional distribution of embeddings.

2D Gaussia Kernel Density Estimation (KDE) Overview KDE is a non-parametric method for estimating the probability density function of a random variable. Given a set of data points $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$, the kernel density estimate at position $\mathbf{x}$ is:

$$\hat{f}(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n K_h(\mathbf{x} - \mathbf{x}_i)$$

where K_h is a kernel function (in this case the Gaussian kernel)

This reveals regions of high and low embedding density, indicating clustering patterns and feature space coverage.

Angular KDE The distribution of angular positions is analyzed separately:

1. Convert Cartesian coordinates to polar angles: $\theta_i = \arctan 2(y_i, x_i)$
2. Fit Gaussian KDE on angular values: $\theta \in [-\pi, \pi]$
3. Plot density function as a 1D curve

This analysis reveals if embeddings are uniformly distributed or not

4 Experiments

4.1 Data distribution

This section presents the spatial and angular distribution of embeddings extracted from the MovieLens 1M and LastFM datasets. Each figure shows the 2D UMAP projection with KDE contours (top row) and the angular density distribution on the unit circle (bottom row) for different encoder configurations.

4.1.1 MovieLens 1M

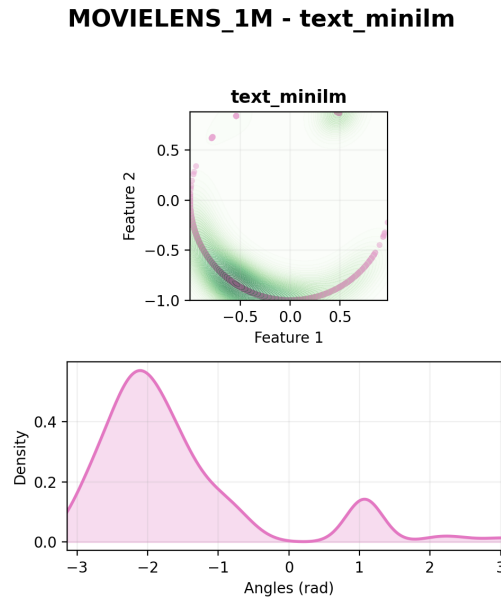


Figure 6: MovieLens 1M - Text embeddings (MiniLM)

MOVIELENS_1M - audio_vggish

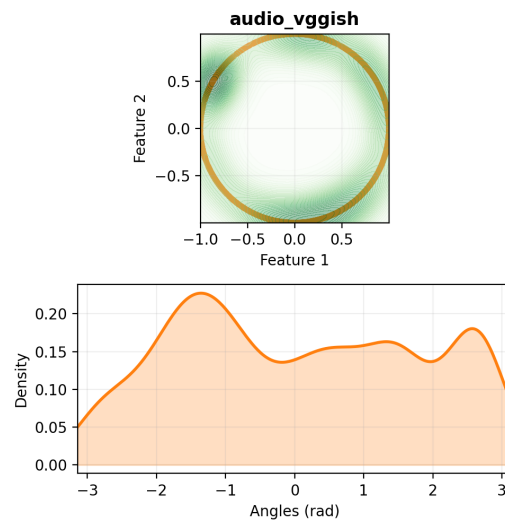


Figure 7: MovieLens 1M - Audio embeddings (VGGish)

MOVIELENS_1M - image_vit

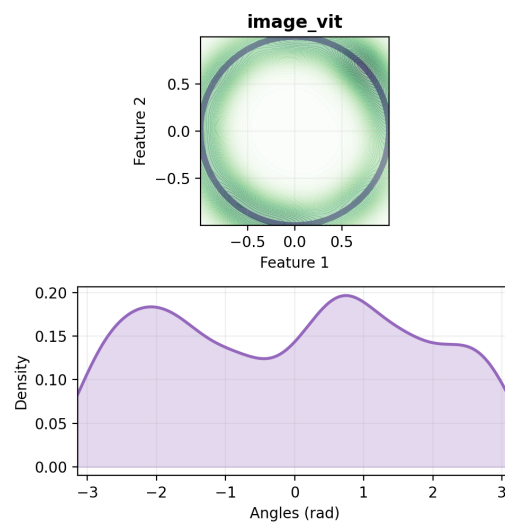


Figure 8: MovieLens 1M - Image embeddings (ViT)

MOVIELENS_1M - text_minilm_image_vit

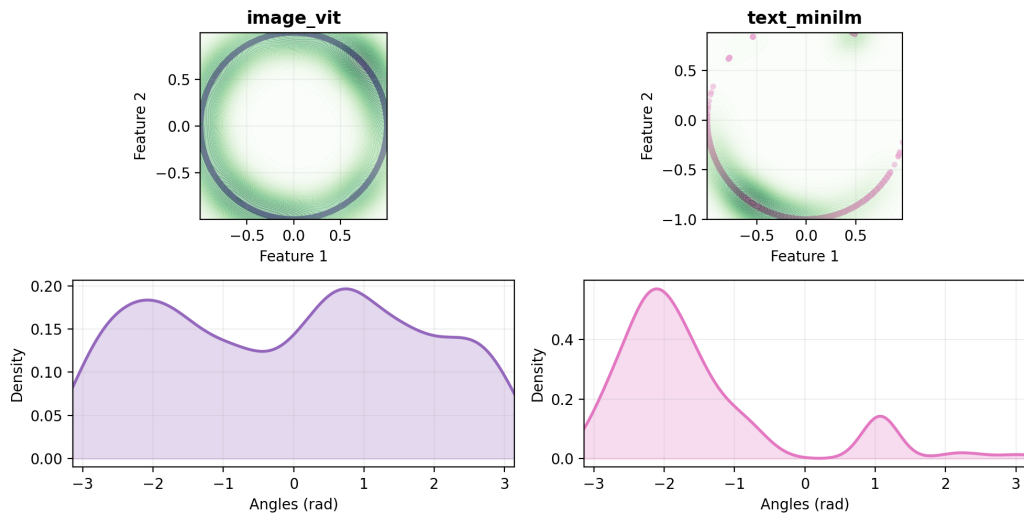


Figure 9: MovieLens 1M - Text (MiniLM) + Image (ViT) embeddings (dis-aligned)

MOVIELENS_1M - text_clip_image_clip

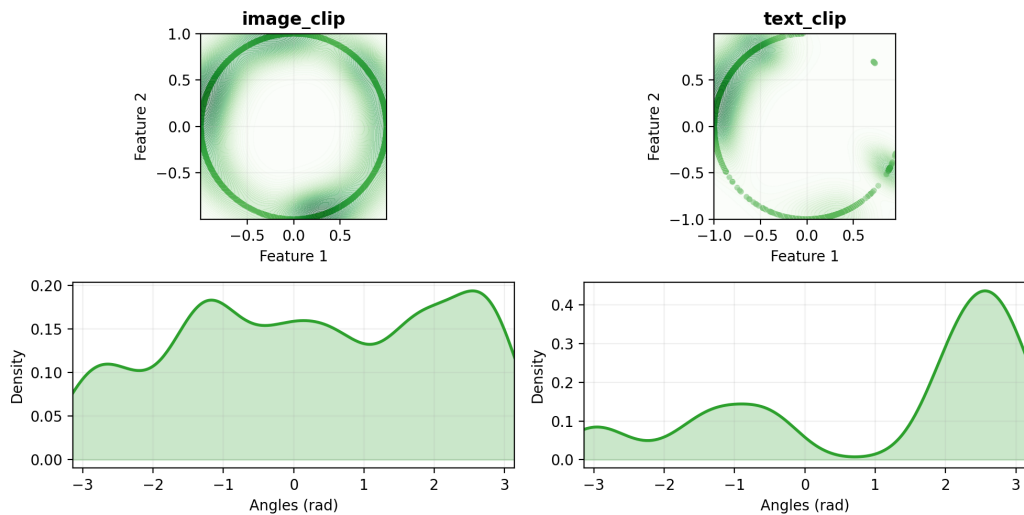


Figure 10: MovieLens 1M - Text + Image embeddings (CLIP aligned)

MOVIELENS_1M - text_minilm_image_vit_audio_vggish

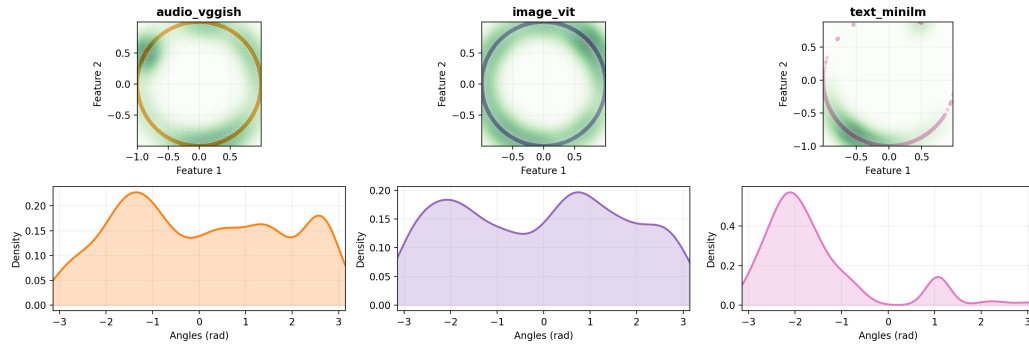


Figure 11: MovieLens 1M - Text (MiniLM) + Image (ViT) + Audio (VG-Gish) embeddings (disaligned)

MOVIELENS_1M - text_clip_image_clip_audio_vggish

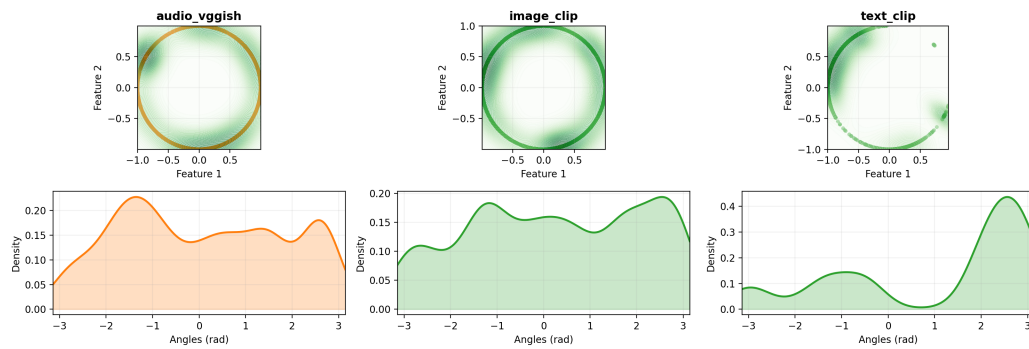


Figure 12: MovieLens 1M - CLIP (Text + Image) + Audio (VGGish) embeddings

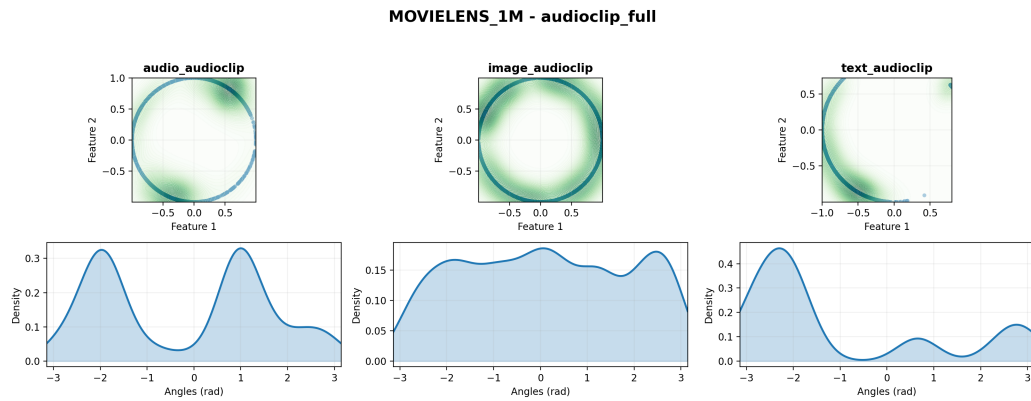


Figure 13: MovieLens 1M - AudioCLIP aligned embeddings (Text + Image + Audio)

4.1.2 LastFM

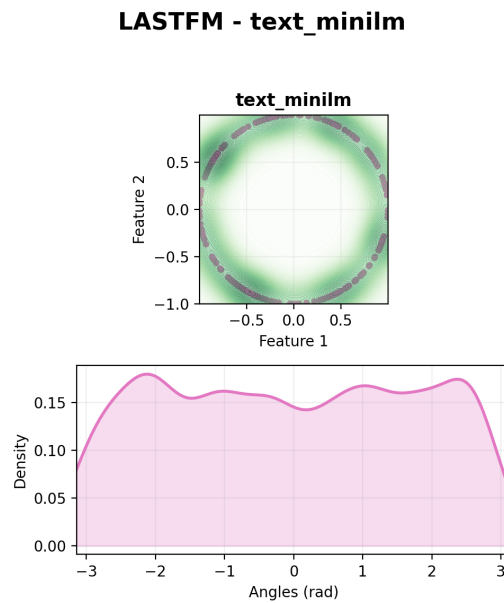


Figure 14: LastFM - Text embeddings (MiniLM)

LASTFM - image_vit

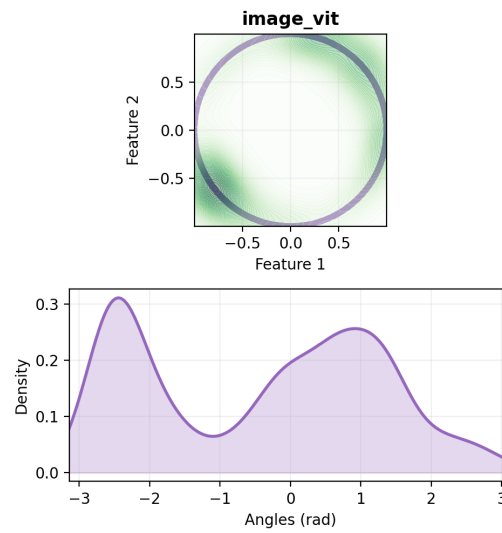


Figure 15: LastFM - Image embeddings (ViT)

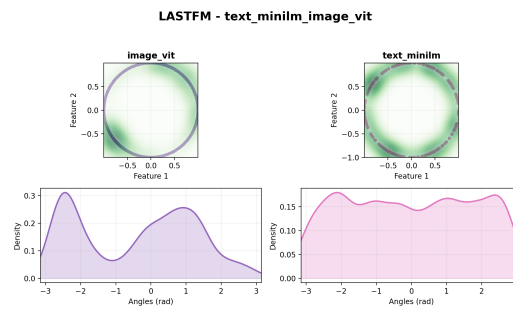


Figure 16: LastFM - Text (MiniLM) + Image (ViT) embeddings (disaligned)

LASTFM - text_clip_image_clip

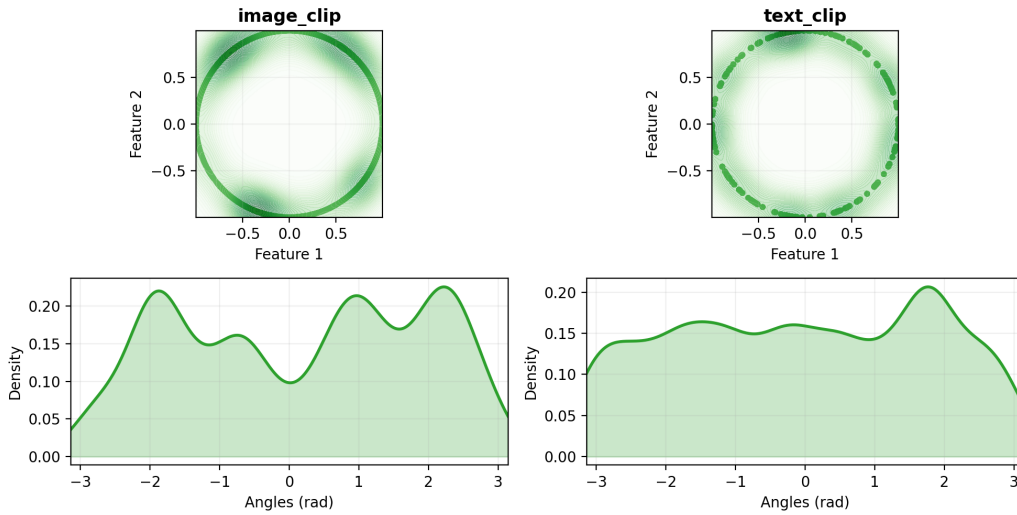


Figure 17: LastFM - Text + Image embeddings (CLIP aligned)

LASTFM - audioclip_full

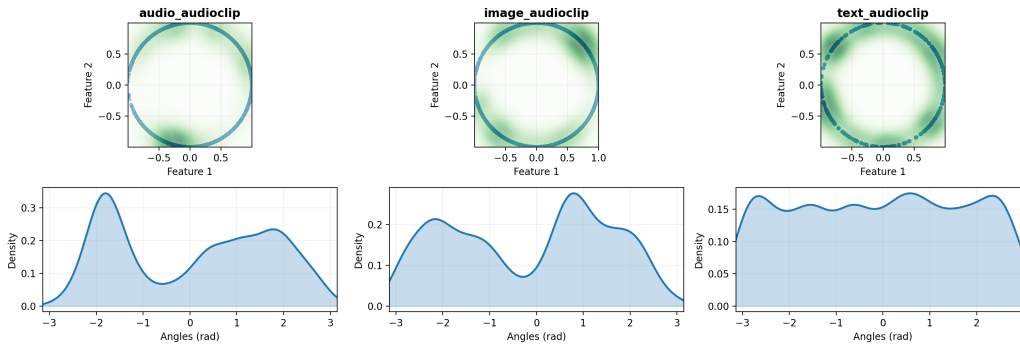


Figure 18: LastFM - AudioCLIP aligned embeddings (Text + Image + Audio)

4.2 Results Overview

This section reports test-set results for several models and multimodal configurations. Tables are grouped by cutoff k : one table for @5, @10, @20 and @50. For each row we report Recall, Precision, NDCG and MAP at the corresponding cutoff.

4.2.1 MovieLens_1M

Feature extractors used Unless explicitly specified in a table row as *CLIP* or *AudioCLIP*, the reported VBPR variants use the following default feature extractors:

- Text: MiniLM (sentence-transformer)
- Image: ViT (Vision Transformer)
- Audio: VGGish-based features

Model	Modality	rec@5	prec@5	ndcg@5	map@5
BPR	None	0.1089	0.1898	0.2096	0.1289
VBPR	Text (MiniLM)	0.1144	0.1999	0.2196	0.1363
VBPR	Audio (VGGish)	0.1147	0.2015	0.2217	0.1374
VBPR	Image (ViT)	0.1148	0.1985	0.2193	0.1356
VBPR	Text+Image (disaligned)	0.1139	0.1972	0.2179	0.1353
VBPR	Text+Image (CLIP)	0.1152	0.1997	0.2205	0.1376
VBPR	Text+Audio+Image (disaligned)	0.1163	0.1986	0.2191	0.1356
VBPR	Text+Audio+Image (AudioCLIP)	0.0940	0.1694	0.1857	0.1125
VBPR	CLIP+audio (disaligned)	0.1158	0.2009	0.2223	0.1384
LATTICE_1	Text (MiniLM)	0.1131	0.1974	0.2169	0.1341
LATTICE_1	Audio (VGGish)	0.108	0.1908	0.2114	0.1312
LATTICE_1	Image (ViT)	0.1118	0.1931	0.2139	0.132
LATTICE_1	Text+Image (disaligned)	0.1146	0.1964	0.2172	0.1343
LATTICE_1	Text+Image (CLIP)	0.1147	0.1963	0.2172	0.1344
LATTICE_1	Text+Audio+Image (AudioCLIP)	0.1133	0.1978	0.2199	0.1375
LATTICE_1	Text+Audio+Image (disaligned)	0.1137	0.1971	0.2194	0.1366
LATTICE_1	CLIP+audio (disaligned)	0.114	0.1988	0.2204	0.1376
LATTICE_3L	Text (MiniLM)	0.114	0.1993	0.2201	0.1368
LATTICE_3L	Audio (VGGish)	0.1129	0.1985	0.2185	0.1357
LATTICE_3L	Image (ViT)	0.1136	0.1997	0.2206	0.1374
LATTICE_3L	Text+Image (disaligned)	0.1135	0.1991	0.2202	0.1373
LATTICE_3L	Text+Image (CLIP)	0.1134	0.1977	0.2189	0.1359
LATTICE_3L	Text+Audio+Image (AudioCLIP)	0.112	0.1988	0.2183	0.1358
LATTICE_3L	Text+Audio+Image (disaligned)	0.1132	0.1971	0.2179	0.1349
LATTICE_3L	CLIP+audio (disaligned)	0.1135	0.1972	0.2177	0.1346
FREEDOM	Text (MiniLM)	0.1133	0.1957	0.2173	0.1347
FREEDOM	Audio (VGGish)	0.1099	0.1927	0.2142	0.1326
FREEDOM	Image (ViT)	0.1106	0.1929	0.2152	0.1337
FREEDOM	Text+Image (disaligned)	0.1111	0.1940	0.2156	0.1335
FREEDOM	Text+Image (CLIP)	0.1106	0.1930	0.2150	0.1331
FREEDOM	Text+Audio+Image (disaligned)	0.1106	0.1938	0.2158	0.1337
FREEDOM	CLIP+audio (disaligned)	0.1103	0.1932	0.2161	0.1343
FREEDOM	Text+Audio+Image (AudioCLIP)	0.1144	0.1984	0.2204	0.1373

Table 1: Metrics at cutoff $k = 5$ (test)

Model	Modality	rec@10	prec@10	ndcg@10	map@10
BPR	None	0.1776	0.1631	0.2159	0.1120
VBPR	Text (MiniLM)	0.1877	0.1713	0.2269	0.1192
VBPR	Audio (VGGish)	0.1876	0.1722	0.2278	0.1194
VBPR	Image (ViT)	0.1881	0.1720	0.2280	0.1200
VBPR	Text+Image (disaligned)	0.1895	0.1725	0.2279	0.1199
VBPR	Text+Image (CLIP)	0.1911	0.1745	0.2300	0.1211
VBPR	Text+Audio+Image (disaligned)	0.1876	0.1718	0.2271	0.1195
VBPR	Text+Audio+Image (AudioCLIP)	0.1570	0.1476	0.1922	0.0969
VBPR	CLIP+audio (disaligned)	0.1898	0.1734	0.2299	0.1211
LATTICE_1	Text (MiniLM)	0.1848	0.1693	0.2238	0.117
LATTICE_1	Audio (VGGish)	0.1806	0.1676	0.2203	0.1153
LATTICE_1	Image (ViT)	0.1835	0.1677	0.2221	0.1158
LATTICE_1	Text+Image (disaligned)	0.1857	0.1689	0.2247	0.1181
LATTICE_1	Text+Image (CLIP)	0.1856	0.1694	0.2248	0.118
LATTICE_1	Text+Audio+Image (AudioCLIP)	0.1883	0.1721	0.2284	0.1206
LATTICE_1	Text+Audio+Image (disaligned)	0.1861	0.1702	0.2267	0.1194
LATTICE_1	CLIP+audio (disaligned)	0.1876	0.1715	0.2279	0.1202
LATTICE_3L	Text (MiniLM)	0.1878	0.1721	0.228	0.1199
LATTICE_3L	Audio (VGGish)	0.1877	0.1724	0.2272	0.1194
LATTICE_3L	Image (ViT)	0.1884	0.1738	0.2291	0.1207
LATTICE_3L	Text+Image (disaligned)	0.1876	0.1715	0.2277	0.1201
LATTICE_3L	Text+Image (CLIP)	0.1877	0.1721	0.2275	0.1198
LATTICE_3L	Text+Audio+Image (AudioCLIP)	0.1875	0.1725	0.2266	0.1188
LATTICE_3L	Text+Audio+Image (disaligned)	0.1848	0.171	0.2257	0.1183
LATTICE_3L	CLIP+audio (disaligned)	0.1862	0.1716	0.2259	0.1182
FREEDOM	Text (MiniLM)	0.1836	0.1678	0.2238	0.1175
FREEDOM	Audio (VGGish)	0.1789	0.1649	0.2200	0.1152
FREEDOM	Image (ViT)	0.1792	0.1652	0.2208	0.1159
FREEDOM	Text+Image (disaligned)	0.1795	0.1652	0.2208	0.1157
FREEDOM	Text+Image (CLIP)	0.1796	0.1657	0.2211	0.1158
FREEDOM	Text+Audio+Image (disaligned)	0.1801	0.1658	0.2215	0.1161
FREEDOM	CLIP+audio (disaligned)	0.1802	0.1660	0.2222	0.1166
FREEDOM	Text+Audio+Image (AudioCLIP)	0.1844	0.1688	0.2259	0.1193

Table 2: Metrics at cutoff $k = 10$ (test)

Model	Modality	rec@20	prec@20	ndcg@20	map@20
BPR	None	0.2743	0.1341	0.2400	0.1112
VBPR	Text (MiniLM)	0.2939	0.1425	0.2544	0.1195
VBPR	Audio (VGGish)	0.2913	0.1432	0.2543	0.1193
VBPR	Image (ViT)	0.2913	0.1424	0.2543	0.1200
VBPR	Text+Image (disaligned)	0.2904	0.1422	0.2534	0.1194
VBPR	Text+Image (CLIP)	0.2942	0.1424	0.2554	0.1204
VBPR	Text+Audio+Image (disaligned)	0.2926	0.1426	0.2542	0.1196
VBPR	Text+Audio+Image (AudioCLIP)	0.2466	0.1225	0.2147	0.0957
VBPR	CLIP+audio (disaligned)	0.2936	0.1432	0.2562	0.1208
LATTICE_1	Text (MiniLM)	0.2862	0.1391	0.249	0.1162
LATTICE_1	Audio (VGGish)	0.2805	0.1375	0.2446	0.1139
LATTICE_1	Image (ViT)	0.2828	0.1371	0.2467	0.1151
LATTICE_1	Text+Image (disaligned)	0.2867	0.1392	0.2504	0.1179
LATTICE_1	Text+Image (CLIP)	0.2887	0.1402	0.2513	0.118
LATTICE_1	Text+Audio+Image (AudioCLIP)	0.2892	0.1409	0.2532	0.1196
LATTICE_1	Text+Audio+Image (disaligned)	0.2878	0.1404	0.2522	0.1188
LATTICE_1	CLIP+audio (disaligned)	0.2881	0.1406	0.2528	0.1194
LATTICE_3L	Text (MiniLM)	0.2915	0.1425	0.2545	0.12
LATTICE_3L	Audio (VGGish)	0.2904	0.1417	0.2531	0.1191
LATTICE_3L	Image (ViT)	0.2933	0.143	0.2553	0.1201
LATTICE_3L	Text+Image (disaligned)	0.2929	0.1427	0.255	0.1203
LATTICE_3L	Text+Image (CLIP)	0.2894	0.1416	0.2531	0.1194
LATTICE_3L	Text+Audio+Image (AudioCLIP)	0.2905	0.1423	0.2523	0.1182
LATTICE_3L	Text+Audio+Image (disaligned)	0.2901	0.1421	0.2526	0.1183
LATTICE_3L	CLIP+audio (disaligned)	0.2909	0.1418	0.2524	0.1182
FREEDOM	Text (MiniLM)	0.2791	0.1364	0.2463	0.1156
FREEDOM	Audio (VGGish)	0.2754	0.1350	0.2433	0.1137
FREEDOM	Image (ViT)	0.2757	0.1351	0.2440	0.1144
FREEDOM	Text+Image (disaligned)	0.2767	0.1356	0.2445	0.1145
FREEDOM	Text+Image (CLIP)	0.2773	0.1355	0.2445	0.1144
FREEDOM	Text+Audio+Image (disaligned)	0.2768	0.1354	0.2446	0.1146
FREEDOM	CLIP+audio (disaligned)	0.2770	0.1357	0.2453	0.1150
FREEDOM	Text+Audio+Image (AudioCLIP)	0.2848	0.1382	0.2505	0.1181

Table 3: Metrics at cutoff $k = 20$ (test)

Model	Modality	rec@50	prec@50	ndcg@50	map@50
BPR	None	0.4499	0.0954	0.2991	0.1250
VBPR	Text (MiniLM)	0.4678	0.1001	0.3131	0.1339
VBPR	Audio (VGGish)	0.4675	0.1006	0.3132	0.1335
VBPR	Image (ViT)	0.4654	0.1002	0.3131	0.1344
VBPR	Text+Image (disaligned)	0.4646	0.0999	0.3119	0.1335
VBPR	Text+Image (CLIP)	0.4681	0.1001	0.3141	0.1347
VBPR	Text+Audio+Image (disaligned)	0.4672	0.1001	0.3129	0.1339
VBPR	Text+Audio+Image (AudioCLIP)	0.4176	0.0892	0.2721	0.1082
VBPR	CLIP+audio (disaligned)	0.4677	0.1004	0.3148	0.1351
LATTICE_1	Text (MiniLM)	0.4603	0.098	0.3073	0.13
LATTICE_1	Audio (VGGish)	0.4536	0.0971	0.3025	0.1273
LATTICE_1	Image (ViT)	0.4589	0.0973	0.3061	0.1295
LATTICE_1	Text+Image (disaligned)	0.4649	0.0983	0.3102	0.1322
LATTICE_1	Text+Image (CLIP)	0.4649	0.099	0.3106	0.1323
LATTICE_1	Text+Audio+Image (AudioCLIP)	0.4641	0.0989	0.3116	0.1333
LATTICE_1	Text+Audio+Image (disaligned)	0.4634	0.0989	0.3109	0.1326
LATTICE_1	CLIP+audio (disaligned)	0.4629	0.0986	0.3111	0.1331
LATTICE_3L	Text (MiniLM)	0.4682	0.1003	0.3138	0.1342
LATTICE_3L	Audio (VGGish)	0.467	0.0998	0.3122	0.1331
LATTICE_3L	Image (ViT)	0.4664	0.1002	0.3135	0.1342
LATTICE_3L	Text+Image (disaligned)	0.4675	0.1005	0.314	0.1347
LATTICE_3L	Text+Image (CLIP)	0.4669	0.1002	0.3127	0.1335
LATTICE_3L	Text+Audio+Image (AudioCLIP)	0.4638	0.0997	0.3102	0.1318
LATTICE_3L	Text+Audio+Image (disaligned)	0.4661	0.1003	0.3117	0.1326
LATTICE_3L	CLIP+audio (disaligned)	0.4654	0.0997	0.311	0.1322
FREEDOM	Text (MiniLM)	0.4464	0.0951	0.3017	0.1280
FREEDOM	Audio (VGGish)	0.4469	0.0952	0.3001	0.1265
FREEDOM	Image (ViT)	0.4466	0.0952	0.3007	0.1271
FREEDOM	Text+Image (disaligned)	0.4466	0.0952	0.3007	0.1271
FREEDOM	Text+Image (CLIP)	0.4467	0.0951	0.3006	0.1270
FREEDOM	Text+Audio+Image (disaligned)	0.4471	0.0952	0.3010	0.1273
FREEDOM	CLIP+audio (disaligned)	0.4471	0.0951	0.3015	0.1276
FREEDOM	Text+Audio+Image (AudioCLIP)	0.4550	0.0966	0.3070	0.1312

Table 4: Metrics at cutoff $k = 50$ (test)

4.2.2 LastFM

Model	Modality	rec@5	prec@5	ndcg@5	map@5
BPR	None	0.0655	0.1577	0.1595	0.1414
VBPR	Text (MiniLM)	0.0733	0.1742	0.1763	0.1585
VBPR	Image (ViT)	0.0754	0.1787	0.1813	0.1611
VBPR	Text+Image (disaligned)	0.0768	0.1831	0.1859	0.1660
VBPR	Text+Image (CLIP)	0.0707	0.1691	0.1714	0.1524
VBPR	Text+Audio+Image (AudioCLIP)	0.0694	0.1711	0.1735	0.1531
LATTICE_1	Text (MiniLM)	0.0679	0.1643	0.1657	0.1575
LATTICE_1	Image (ViT)	0.0684	0.1652	0.1664	0.1456
LATTICE_1	Text+Image (disaligned)	0.0692	0.1685	0.1685	0.1564
LATTICE_1	Text+Image (CLIP)	0.0714	0.1721	0.1734	0.1624
LATTICE_1	Text+Audio+Image (AudioCLIP)	0.0701	0.1682	0.1692	0.1559
LATTICE_3L	Text (MiniLM)	0.077	0.184	0.1876	0.1692
LATTICE_3L	Image (ViT)	0.0749	0.1756	0.1791	0.1574
LATTICE_3L	Text+Image (disaligned)	0.0757	0.1825	0.1856	0.1646
LATTICE_3L	Text+Image (CLIP)	0.0726	0.1751	0.1762	0.1564
LATTICE_3L	Text+Audio+Image (AudioCLIP)	0.0712	0.1718	0.1742	0.1538
FREEDOM	Text (MiniLM)	0.0522	0.1358	0.1358	0.1303
FREEDOM	Image (ViT)	0.0517	0.1356	0.1362	0.1305
FREEDOM	Text+Image (disaligned)	0.0527	0.1362	0.1362	0.1306
FREEDOM	Text+Image (CLIP)	0.0522	0.1358	0.1357	0.1303
FREEDOM	Text+Audio+Image (AudioCLIP)	0.0526	0.1358	0.1358	0.1299

Table 5: Metrics at cutoff $k = 5$ (test) for LastFM

Model	Modality	rec@10	prec@10	ndcg@10	map@10
BPR	None	0.1132	0.1404	0.1534	0.1061
VBPR	Text (MiniLM)	0.1268	0.1531	0.1695	0.1200
VBPR	Image (ViT)	0.1298	0.1552	0.1734	0.1214
VBPR	Text+Image (disaligned)	0.1307	0.1560	0.1755	0.1242
VBPR	Text+Image (CLIP)	0.1222	0.1488	0.1646	0.1149
VBPR	Text+Audio+Image (AudioCLIP)	0.1196	0.1482	0.1636	0.1136
LATTICE_1	Text (MiniLM)	0.1222	0.1492	0.1627	0.1195
LATTICE_1	Image (ViT)	0.1193	0.1475	0.1608	0.1109
LATTICE_1	Text+Image (disaligned)	0.1233	0.1516	0.164	0.1184
LATTICE_1	Text+Image (CLIP)	0.1244	0.1537	0.1676	0.1216
LATTICE_1	Text+Audio+Image (AudioCLIP)	0.1231	0.1508	0.1643	0.1173
LATTICE_3L	Text (MiniLM)	0.1306	0.1568	0.1765	0.1258
LATTICE_3L	Image (ViT)	0.1268	0.1526	0.1707	0.1191
LATTICE_3L	Text+Image (disaligned)	0.1258	0.1535	0.1722	0.1209
LATTICE_3L	Text+Image (CLIP)	0.1232	0.1507	0.1667	0.116
LATTICE_3L	Text+Audio+Image (AudioCLIP)	0.1238	0.151	0.1669	0.116
FREEDOM	Text (MiniLM)	0.0988	0.1225	0.1321	0.0954
FREEDOM	Image (ViT)	0.0960	0.1214	0.1312	0.0952
FREEDOM	Text+Image (disaligned)	0.0986	0.1224	0.1322	0.0954
FREEDOM	Text+Image (CLIP)	0.0986	0.1225	0.1320	0.0953
FREEDOM	Text+Audio+Image (AudioCLIP)	0.0989	0.1227	0.1323	0.0953

Table 6: Metrics at cutoff $k = 10$ (test) for LastFM

Model	Modality	rec@20	prec@20	ndcg@20	map@20
BPR	None	0.1890	0.1161	0.1703	0.0998
VBPR	Text (MiniLM)	0.1990	0.1218	0.1831	0.1119
VBPR	Image (ViT)	0.2086	0.1265	0.1899	0.1142
VBPR	Text+Image (disaligned)	0.2054	0.1243	0.1896	0.1153
VBPR	Text+Image (CLIP)	0.2009	0.1225	0.1820	0.1082
VBPR	Text+Audio+Image (AudioCLIP)	0.1915	0.1189	0.1766	0.1042
LATTICE_1	Text (MiniLM)	0.2001	0.1241	0.1804	0.1119
LATTICE_1	Image (ViT)	0.1998	0.1228	0.1791	0.1047
LATTICE_1	Text+Image (disaligned)	0.2017	0.1248	0.1813	0.1108
LATTICE_1	Text+Image (CLIP)	0.2038	0.1254	0.1847	0.1138
LATTICE_1	Text+Audio+Image (AudioCLIP)	0.204	0.1255	0.1833	0.1114
LATTICE_3L	Text (MiniLM)	0.2032	0.1239	0.1894	0.1159
LATTICE_3L	Image (ViT)	0.2041	0.124	0.1872	0.1117
LATTICE_3L	Text+Image (disaligned)	0.2082	0.126	0.1898	0.1131
LATTICE_3L	Text+Image (CLIP)	0.2079	0.1264	0.1866	0.1106
LATTICE_3L	Text+Audio+Image (AudioCLIP)	0.2072	0.1264	0.1864	0.1104
FREEDOM	Text (MiniLM)	0.1653	0.1027	0.1466	0.0876
FREEDOM	Image (ViT)	0.1627	0.1020	0.1454	0.0862
FREEDOM	Text+Image (disaligned)	0.1650	0.1026	0.1466	0.0877
FREEDOM	Text+Image (CLIP)	0.1654	0.1027	0.1467	0.0876
FREEDOM	Text+Audio+Image (AudioCLIP)	0.1644	0.1023	0.1463	0.0875

Table 7: Metrics at cutoff $k = 20$ (test) for LastFM

Model	Modality	rec@50	prec@50	ndcg@50	map@50
BPR	None	0.3318	0.0814	0.2343	0.1211
VBPR	Text (MiniLM)	0.3494	0.0858	0.2509	0.1344
VBPR	Image (ViT)	0.3566	0.0867	0.2562	0.1367
VBPR	Text+Image (disaligned)	0.3484	0.0853	0.2543	0.1374
VBPR	Text+Image (CLIP)	0.3539	0.0867	0.2508	0.1314
VBPR	Text+Audio+Image (AudioCLIP)	0.3383	0.0835	0.2422	0.1257
LATTICE_1	Text (MiniLM)	0.349	0.0854	0.2464	0.1338
LATTICE_1	Image (ViT)	0.3415	0.0839	0.2427	0.1258
LATTICE_1	Text+Image (disaligned)	0.3445	0.0842	0.2446	0.1316
LATTICE_1	Text+Image (CLIP)	0.3449	0.0843	0.2477	0.1349
LATTICE_1	Text+Audio+Image (AudioCLIP)	0.3481	0.0853	0.2475	0.1329
LATTICE_3L	Text (MiniLM)	0.3477	0.085	0.2543	0.1377
LATTICE_3L	Image (ViT)	0.3548	0.0866	0.2549	0.1345
LATTICE_3L	Text+Image (disaligned)	0.3484	0.0851	0.2529	0.1344
LATTICE_3L	Text+Image (CLIP)	0.347	0.0849	0.2492	0.1319
LATTICE_3L	Text+Audio+Image (AudioCLIP)	0.3485	0.0853	0.2501	0.1324
FREEDOM	Text (MiniLM)	0.2990	0.0745	0.2070	0.1070
FREEDOM	Image (ViT)	0.3016	0.0750	0.2078	0.1062
FREEDOM	Text+Image (disaligned)	0.2997	0.0745	0.2074	0.1071
FREEDOM	Text+Image (CLIP)	0.2991	0.0745	0.2070	0.1070
FREEDOM	Text+Audio+Image (AudioCLIP)	0.2994	0.0745	0.2072	0.1070

Table 8: Metrics at cutoff $k = 50$ (test) for LastFM

5 Discussion

This section analyzes the experimental results across both datasets, examining the impact of aligned vs. disaligned multimodal embeddings on recommendation performance and the geometric properties of the embedding spaces.

5.1 Data Distribution Analysis

To understand the impact of feature alignment, we analyzed the spatial and angular distributions of the embeddings projected onto a 2D space using UMAP.

MovieLens 1M The geometric analysis of MovieLens 1M embeddings reveals a critical anomaly in the fully aligned AudioCLIP model. As shown in Figure 13, the density distributions of the *audio* and *image* modalities are nearly identical, effectively collapsing into overlapping regions, while the *text* distribution remains distinct. This suggests that AudioCLIP’s aggressive alignment strategy forces visual and acoustic features to lose their modality-specific discriminative power. Conversely, the partially aligned CLIP embeddings (Text+Image) in Figure 10 maintain a shared structure without complete collapse, showing a more balanced integration compared to the disaligned features in Figure 9.

LastFM For LastFM, the distribution patterns differ significantly. Comparing the disaligned embeddings (Figure 16) with the aligned CLIP embeddings (Figure 17), we observe that the disaligned space exhibits a higher number of angular peaks. This indicates a greater diversity in directional orientations for disaligned features. This suggests that preserving the unique geometric structure of each modality (disalignment) allows the model to capture a broader range of user preferences in the music domain, whereas alignment tends to smooth out these distinctions.

5.2 Performance Analysis

We evaluated the models (VBPR, LATTICE, FREEDOM) across varying cutoff thresholds ($k \in \{5, 10, 20, 50\}$). The results highlight distinct trends regarding model architecture and the efficacy of multimodal alignment.

5.2.1 Model Comparison

Across both datasets, multimodal approaches consistently outperform the unimodal BPR baseline, confirming the value of side information.

- **VBPR Dominance:** VBPR consistently achieves the highest performance metrics compared to LATTICE and FREEDOM. For instance, on MovieLens 1M at $k = 5$ (Table 1), VBPR with CLIP+Audio (disaligned) reaches a Recall of 0.1158, surpassing the best LATTICE configuration (0.1147) and FREEDOM (0.1144).
- **LATTICE vs. FREEDOM:** LATTICE generally performs competitively, particularly with 3 layers (LATTICE_3L), often outperforming the 1-layer variant and FREEDOM, but remains slightly below VBPR’s peak performance as seen in Table 1 and Table 2.

5.2.2 Impact of Feature Alignment on MovieLens 1M

On the MovieLens 1M dataset, alignment generally yields positive results, with specific caveats regarding AudioCLIP.

- **Benefit of CLIP Alignment:** Configurations using CLIP-aligned (Text+Image) features outperform those using disaligned (MiniLM+ViT) features. For example, in Table 1, VBPR with CLIP achieves a Recall@5 of 0.1152 compared to 0.1139 for the disaligned counterpart. This validates that semantic alignment between movie posters and textual descriptions enhances recommendation quality.
- **The AudioCLIP Failure:** Consistent with the geometric ”collapse” observed in Figure 13, the fully aligned AudioCLIP configuration performs significantly worse than all other methods. At $k = 5$ (Table 1), VBPR with AudioCLIP drops to a Recall of 0.0940, the lowest among multimodal variants. This confirms that the lack of discriminative variance in the AudioCLIP embedding space harms the model’s ability to rank items effectively.
- **Best Configuration:** The optimal performance is achieved by combining aligned CLIP features (Text+Image) with disaligned Audio features (VGGish). This setup (VBPR CLIP+audio disaligned) achieves the highest Recall@5 of 0.1158 and NDCG@5 of 0.2223 (Table 1).

5.2.3 Impact of Feature Alignment on LastFM

The LastFM dataset exhibits an inverse trend regarding alignment, favoring modality-specific feature spaces.

- **Superiority of Disaligned Features:** Unlike MovieLens, the disaligned Text+Image configuration consistently outperforms the aligned CLIP configuration. At $k = 5$ (Table 5), VBPR (Text+Image disaligned) achieves a Recall of 0.0768, significantly higher than the CLIP version at 0.0707. This aligns with the geometric observation that disaligned features retain more directional diversity.
- **AudioCLIP Stability:** While AudioCLIP failed on MovieLens, it remains competitive on LastFM. At $k = 10$ (Table 6), LATTICE_1 using AudioCLIP achieves a Recall of 0.1231, comparable to the disaligned Text+Image score of 0.1233. This suggests that the structural collapse observed in MovieLens is less severe or less detrimental for the music domain.

5.2.4 The Role of Audio Modality

The inclusion of audio features shows mixed effects depending on the integration strategy. While VGGish features (disaligned) generally enhance performance when added to text and image (as seen in the "CLIP+audio (disaligned)" rows in Table 1), the joint training of audio within the AudioCLIP framework proves detrimental for movies but neutral for music. This indicates that while audio provides valuable signal, forcing it into a shared semantic space with text and images via aggressive contrastive learning may be suboptimal for recommendation tasks where orthogonal features add more value.

6 Conclusion and Future Work

6.1 Summary of Findings

This work investigated the impact of using aligned versus disaligned multimodal features in recommender systems, focusing on the MovieLens 1M and LastFM datasets. Through extensive experiments comparing standard pretrained features (MiniLM, ViT, VGGish) against contrastively aligned representations (CLIP, AudioCLIP), we addressed the research question of whether semantic alignment enhances recommendation performance

Our findings reveal that the effectiveness of feature alignment is highly domain-dependent:

- **MovieLens 1M:** In the movie domain, a hybrid approach yields the best results. The partial alignment provided by CLIP (Text+Image) improves performance over disaligned features, confirming that semantically connecting visual and textual data is beneficial. However, the inclusion of audio requires a disaligned approach (VGGish), as the fully aligned AudioCLIP model suffered from a geometric "feature collapse," leading to inferior performance.
- **LastFM:** In the music domain, preserving the distinct geometric structure of each modality proved superior. Disaligned features consistently outperformed aligned ones, suggesting that the directional diversity of independent feature spaces captures the nuances of music preferences better than a forced shared semantic space.
- **Model Performance:** Among the evaluated architectures, VBPR consistently demonstrated robustness and superior performance compared to more complex graph-based models like LATTICE and FREEDOM across most configurations. This confirms the result obtained by Zixuan Yi et al. [10] about LATTICE performance with aligned features

6.2 Interplay between Model Complexity and Data Alignment

By cross-referencing the experimental results regarding model performance (Section 5.2) and data distribution analysis (Section 5.1), we can derive specific heuristics concerning the relationship between model choice and feature alignment. The findings suggest that the optimal configuration depends heavily on the intrinsic geometry of the domain data.

1. Data-Driven Strategy: Domain Semantics dictate Alignment.

When the dataset is characterized by strong semantic correlations between modalities (e.g., visual posters and textual descriptions in *MovieLens 1M*), it is optimal to utilize **aligned features** (e.g., CLIP). In this context, the semantic overlap enhances the recommendation signal. However, when the domain is abstract or relies on high directional diversity (e.g., *LastFM*), it is preferable to use **disaligned features**. In the latter case, forcing alignment smooths out the unique geometric nuances required to capture user preferences.

2. Model-Driven Strategy: Architecture implies Feature Requirements. When employing robust, direct-projection architectures like **VBPR**, the system demonstrates high resilience and consistently outperforms more complex Graph Neural Networks (GNNs) like *LATTICE* and *FREEDOM* across most configurations. Consequently:

- If the chosen model is a complex GNN (e.g., *LATTICE*), it is crucial to avoid aggressively aligned data (such as *AudioCLIP*). The "feature collapse" observed in fully aligned spaces deprives the graph convolution layers of the necessary discriminative variance, leading to inferior performance compared to simpler models.
- Conversely, if the data representation is naturally disaligned or preserves high geometric diversity, a simpler model like VBPR is sufficient to exploit these features effectively, rendering the additional computational complexity of GNNs unnecessary.

In summary, increased model complexity does not compensate for poor feature representation. Instead, simpler models combined with domain-appropriate feature alignment (aligned for semantic, disaligned for abstract) yield the superior performance.

6.3 Future Work

Based on the limitations and insights observed in this study, several directions for future research emerge:

Refining Audio Alignment in RecSys The significant performance drop and geometric collapse observed with AudioCLIP suggest that current contrastive alignment strategies may be too aggressive for audio data in recommendation contexts. Future work should investigate "softer" alignment techniques or regularization methods that encourage a shared semantic space

without sacrificing the intra-modality discriminative power essential for ranking items.

Domain-Adaptive Fusion Strategies The contrast between results in MovieLens and LastFM highlights the need for adaptive fusion mechanisms that dynamically weight the importance of feature according to domain characteristics. Developing models that can learn to select between aligned and disaligned representations based on dataset properties could lead to more universally effective recommender systems.

References

- [1] Xi Chen, Yangsiyi Lu, Yuehai Wang, and Jianyi Yang. Cmbf: Cross-modal-based fusion recommendation algorithm. *Sensors*, 21(16), 2021.
- [2] Andrey Guzhov, Federico Raue, Jörn Hees, and Andreas Dengel. Audioclip: Extending clip to image, text and audio. In *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 976–980. IEEE, 2022.
- [3] Haigen Hu, Xiaoyuan Wang, Yan Zhang, Qi Chen, and Qiu Guan. A comprehensive survey on contrastive learning. *Neurocomputing*, 610:128645, 2024.
- [4] Peishan Li, Weixiao Zhan, Lutao Gao, Shuran Wang, and Linnan Yang. Multimodal recommendation system based on cross self-attention fusion. *Systems*, 13(1), 2025.
- [5] Pasquale Lops, Marco de Gemmis, and Giovanni Semeraro. *Content-based Recommender Systems: State of the Art and Trends*, pages 73–105. Springer US, Boston, MA, 2011.
- [6] Nitin Mishra, Saumya Chaturvedi, Aanchal Vij, and Sunita Tripathi. Research problems in recommender systems. In *Journal of Physics: Conference Series*, volume 1717, page 012002. IOP publishing, 2021.
- [7] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR, 2021.
- [8] Yu Rong, Xiao Wen, and Hong Cheng. A monte carlo algorithm for cold start recommendation. In *Proceedings of the 23rd international conference on World wide web*, pages 327–336, 2014.
- [9] Giuseppe Spillo, Elio Musacchio, Cataldo Musto, Marco de Gemmis, Pasquale Lops, and Giovanni Semeraro. See the movie, hear the song, read the book: Extending movielens-1m, last.fm-2k, and dbbook with multimodal data. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems, RecSys '25*, page 847–856, New York, NY, USA, 2025. Association for Computing Machinery.

- [10] Zixuan Yi, Zijun Long, Iadh Ounis, Craig Macdonald, and Richard Mccreadie. Large multi-modal encoders for recommendation. *arXiv preprint arXiv:2310.20343*, 2023.